OblivRec: Private Serving and Delivery

Mid-Term Technical Report, CS670

Mainak Sarkar (230619) Shrasti Dwivedi (230982) Aditya Anand (230065)
Shravan Agrawal (230984)

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur
12 September 2026

Abstract

This report records the first implementation slice of OblivRec, a recommender that composes the three-party private training core of NUDGE with the private delivery layer of PIRSONA. The slice covers private serving and delivery against a model trained in the clear, which is stage S1 of the staging fixed in the requirements. We report a distributed point function written from the Boyle–Gilboa–Ishai construction, a DPF-based private information retrieval read layer over the MovieLens catalogue, a cleartext power-iteration factorisation used as the quality oracle, and an experiment quantifying what an observer learns from watching which items a user fetches. Private training itself is out of scope here and is scheduled for the following phase.

Contents

1 Scope of this slice **2**
1.1 What is deliberately absent . . . 2

2 The cleartext oracle, and how many power iterations are needed **2**
2.1 Construction . . . 2
2.2 Quality is flat in the iteration count . . . 3
2.3 Why this matters for private training . . . 3

3 The distributed point function **4**
3.1 Why the keys are small . . . 4
3.2 The sign convention . . . 4
3.3 Where the key material comes from . . . 4
3.4 Testing: the structural invariant, checked at every node . . . 5
3.5 Correctness results . . . 5
3.6 The wire format, and a bug found by testing it . . . 6
3.7 Hardened verification . . . 6

1 Scope of this slice

The full system has two halves. Private *training* learns a factorisation of a rating matrix that no server may see, and private *delivery* fetches the recommended item without revealing which item it was. The requirements document fixes the build order as S1, then S2, then S3: serving and delivery first, then training, then the composition. S1 is first because it is lower risk, it demonstrates something end to end early, and the distributed point function it needs is a prerequisite for the non-linear gates that private training will need later.

This report covers S1 only. The model is trained *in the clear* by the Python oracle of Section 2, and everything downstream of that model is private. Private training is the subject of the next phase and no claim is made about it here.

1.1 What is deliberately absent

Three components specified in the design draft are not built in this slice, and each was cut for a stated reason rather than for lack of time.

A distributed comparison function is not implemented. Its only consumers are the truncation and normalisation protocols, both of which belong to private training. The design draft asserts that a distributed point function and a distributed comparison function are the same primitive. They are not, and the reference implementation of Nudge keeps them in separate modules, which corroborates the distinction.

Oblivious top-k selection is not implemented. The user reconstructs the score vector and selects the top k locally, so no server learns the selection regardless of whether an oblivious selector exists. The apparatus would protect something that is already protected.

Network emulation is not used. The development machine has neither Docker nor `tc netem`, so every measurement in this report is taken on the `local` profile and is labelled as such. Wide-area numbers are deferred rather than estimated.

2 The cleartext oracle, and how many power iterations are needed

Every private component in this project is checked against a cleartext twin. The factorisation oracle is therefore the first thing built, before any cryptography, and it doubles as baseline B1, the speed-of-light reference against which recommendation quality is measured.

2.1 Construction

The oracle implements ApproxFactor as Nudge states it, rather than calling a singular value decomposition. This is deliberate. The private implementation in the next phase must reproduce this procedure step by step under secret sharing, so the reference has to match the *structure* of the protocol and not merely its output. Concretely, for each of d components a random vector is repeatedly multiplied by $\mathbf{U}^\top\mathbf{U}$, made orthogonal to the components already found, and normalised. Each converged component is then revealed in the clear and becomes a row of $\mathbf{B}$.

Revealing each row before computing the next is what keeps the orthogonalisation cheap, because it is then a Gram–Schmidt step against public vectors. It is also the reason the item embedding matrix is public to every server, which is the premise of the leakage analysis in this project.

A singular value decomposition is still computed, but as a *cross-check* rather than as the implementation. Power iteration on $\mathbf{U}^\top\mathbf{U}$ converges to the top-d right singular subspace, so the

largest principal angle between the two subspaces should approach zero. That check costs two lines and it caught a genuine question on the first day.

2.2 Quality is flat in the iteration count

At the default of $\ell = 10$ inner rounds the subspace angle is 9.66×10^{-1} radians, which is nowhere near zero. Sweeping ℓ resolves what that means.

Table 1: MovieLens-100K, split `u1`, $d = 16$. The subspace angle falls by five orders of magnitude while recommendation quality does not move. Raw data in `bench/results/oracle_ell_sweep.jsonl`.

ℓ	subspace angle (rad)	nDCG@20	Recall@20
10	9.659×10^{-1}	0.4536	0.4693
25	4.170×10^{-1}	0.4545	0.4677
50	1.834×10^{-1}	0.4525	0.4650
100	6.966×10^{-2}	0.4523	0.4662
200	3.604×10^{-3}	0.4526	0.4664
400	6.652×10^{-6}	0.4526	0.4664

Two conclusions follow, and they are separate.

The first is that the implementation is correct. The angle falls monotonically to 6.7×10^{-6} radians, so the procedure does converge to the right subspace. The large angle at $\ell = 10$ is slow convergence rather than a defect. The spectrum explains the rate: the ratio of the sixteenth to the seventeenth squared singular value is 1.032, a gap of three percent, and power iteration converges as a power of that ratio. The first component is easy, with a ratio of 6.44. The sixteenth is not.

This distinction was nearly lost. A failing cross-check invites the reaction of removing the cross-check, which would have converted a correct implementation into a silently wrong one.

The second conclusion is the useful one. Across a fortyfold increase in ℓ, nDCG@20 stays between 0.4523 and 0.4545, a spread of about half a percent with no monotone trend. Ranking does not require the singular vectors, only a subspace good enough that the ordering of scores is stable.

2.3 Why this matters for private training

Under two-out-of-three replicated sharing the matrix–vector products are non-interactive and therefore free. Truncation and normalisation are the entire communication cost of training, and they run once per inner iteration, so communication is linear in ℓ.

The measurement therefore says that private training can run at $\ell = 10$ and the fortyfold saving in communication costs nothing in recommendation quality. That is a design decision resting on evidence rather than on the default value in the design draft, and it also discharges an instruction the draft had already given and which had not been acted on, namely to measure the residual against the cleartext oracle and report iterations against quality.

The caveats are stated plainly. This is one dataset, one split, one seed and one embedding dimension. The flatness may not survive at MovieLens-1M or at $d = 32$, and it will be re-measured before the final report relies on it. The metric uses binary relevance at a rating of four or above, so these numbers are a within-project baseline and are not directly comparable to the figures NUDGE reports on Netflix data under its own convention.

3 The distributed point function

A point function $f_{\alpha,\beta}$ takes the value β at $x = \alpha$ and zero everywhere else. A $(2, 2)$-distributed point function splits it into two keys, each of which individually hides both α and β, such that the two evaluations recombine to $f_{\alpha,\beta}(x)$. The construction is that of Boyle, Gilboa and Ishai, and it is written by us rather than imported, because it is the component a viva will probe hardest and because the same implementation serves both halves of this project: the private read layer in Section **??**, and the function secret sharing gates that private training will need in the next phase.

3.1 Why the keys are small

The construction is a binary tree of pseudorandom seeds. Each key holds one initial seed and one correction word per level, so key size is *logarithmic* in the domain while a naive one-hot query is linear. Table 2 gives measured sizes from the serialiser rather than from the closed form, since the closed form is what the design draft got wrong.

Table 2: Measured DPF key size against the naive alternative. The packed format is $1 + 4 + 16 + 18\,\text{db} + \lceil b/8 \rceil$ bytes, and `SizeBytes()` is asserted equal to the actual serialised length.

domain bits	n	key, $b = 64$	key, $b = 128$	naive one-hot	compression
10	1,024	209 B	217 B	8,192 B	39×
11	2,048	227 B	235 B	16,384 B	72×
12	4,096	245 B	253 B	32,768 B	134×
16	65,536	317 B	325 B	524,288 B	1654×

The design draft claims roughly 260 bytes at $n = 4096$ against 32 KB naive. The 32 KB figure is right. The 260-byte figure is not: the measured value is **245 bytes**, and the discrepancy comes from the draft's formula omitting both the initial seed and the self-describing header, while simultaneously evaluating at 16 domain bits rather than the 12 that $n = 4096$ implies. The number in this report is measured by a test that prints it, which is why the error surfaced.

3.2 The sign convention

Textbook presentations of this construction define $\text{Eval}_b = (-1)^b\big(\text{Convert}(s) + t \cdot \text{CW}_{\text{last}}\big)$, which yields the *sum* convention $\text{Eval}_0(x) + \text{Eval}_1(x) = f(x)$. The specification for this project instead states the invariant as a *difference*. We therefore omit the $(-1)^b$ factor and return $\text{Convert}(s) + t \cdot \text{CW}_{\text{last}}$ directly, giving

$$\text{Eval}(k_0, x) - \text{Eval}(k_1, x) = \begin{cases} \beta & x = \alpha \\ 0 & \text{otherwise.} \end{cases}$$

The two conventions are algebraically identical and differ only in where the negation is placed. The reason to record the choice explicitly is that getting it backwards makes every test fail in a way that resembles a tree traversal bug rather than a sign error, which is an expensive place to spend a day.

3.3 Where the key material comes from

The initial seeds are the entire secret. Every correction word in a key is public-looking by construction, and the security argument rests on an adversary being unable to guess or reconstruct the two initial seeds.

The first version of this code seeded a Mersenne Twister and used its output as seed material. That was a genuine break rather than an infelicity. A Mersenne Twister is fully reconstructible from 624 consecutive outputs, so an adversary who observed enough key material could rebuild the tree and recover α, which is precisely the value the construction exists to hide. The seeding also supplied roughly 64 bits of entropy against a claimed security parameter of $\lambda = 128$.

Key material is now drawn from the operating system generator through the Windows CNG interface. The failure policy is to abort rather than fall back to a weaker source, because a silent fallback would leave a broken guarantee indistinguishable from a working one. The accompanying tests check the properties that actually fail in practice: that successive draws differ across two thousand samples, that output is not stuck at a constant, that the bit balance and byte spread are sane, and that odd request lengths neither overrun the caller's buffer nor leave part of it unwritten. None of that establishes cryptographic quality, which no test suite can, and the report does not claim otherwise.

3.4 Testing: the structural invariant, checked at every node

A black-box test reports only that a leaf is wrong. The characteristic failure of this construction is a mistaken control-bit update, which stays invisible until enough levels have accumulated for it to change an output, and therefore presents as "correct at two domain bits, wrong at eight". Bisecting that from leaf values is slow.

The test suite therefore walks *both* parties' trees in lockstep and asserts the underlying structural invariant at every internal node. Writing (s_b, t_b) for party b's state:

On the path to α	$s_0 \neq s_1$ and $t_0 \oplus t_1 = 1$
Off the path	$s_0 = s_1$ and $t_0 \oplus t_1 = 0$

Off the path the two parties hold identical state, which is precisely why their contributions cancel. On the path they differ and their control bits disagree, which is what lets the final correction word inject β at exactly one leaf. Every correctness property of the scheme follows from these two lines, so a violation names the exact level and node at which the construction first went wrong.

This checker was written *before* the generator was trusted, and both were exercised together on the first run.

3.5 Correctness results

The structural invariant holds at every node for every α in every domain up to eight bits, and at sampled α up to eleven bits.

The difference invariant is checked against a brute-force point function oracle for every α and every x: exhaustively to ten domain bits on the 64-bit ring and nine on the 128-bit ring, then for all x at sampled α at eleven and twelve bits. Edge cases that off-by-one errors land on are covered explicitly, namely α at the first and last index, $\beta = 0$ where everything must cancel, and $\beta = -1$ where the ring wraps. A separate check confirms that $\beta = 1$ reconstructs exactly, since the read layer depends on that with no tolerance for an error term.

Traversal is implemented once and shared by single-point evaluation, whole-domain evaluation and the test checker, so there is one traversal implementation to be wrong rather than three.

3.6 The wire format, and a bug found by testing it

Whole-domain evaluation and key serialisation were written alongside the header, since the header is a frozen cross-component contract and had to be complete, but they were initially exercised only indirectly. Testing them afterwards found one real defect.

Deserialisation accepted a party byte outside $\{0, 1\}$. This is not cosmetic. Evaluation begins with t set to the party index and the traversal step branches on whether t is non-zero, so a party byte of, say, 7 is truthy and the key would *silently evaluate as party one*. A corrupted or malicious key therefore produced a wrong reconstruction with no error raised anywhere. Since keys arrive over the wire, the fix is to validate on entry rather than to trust the sender. The domain size is now validated on the same principle, before it is used either to size an allocation or to multiply out an expected length.

The tests that found it are retained as regression tests rather than discarded. They cover round-tripping on both rings, including that a full pair of keys still reconstructs β after both have crossed the wire, and they check that truncated, over-length, empty, and malformed-header inputs are all rejected. Whole-domain and single-point evaluation are asserted to agree at every index, and whole-domain evaluation is required to reject a wrongly sized output buffer rather than overrun it.

Two further checks were run and left in place. The decimal formatter for the 128-bit ring writes thirty-nine digits into a forty-byte buffer, so it has zero margin at the maximum representable value and that value is now pinned by a test. And the ranking metric used for every quality number in this report was cross-checked against an independent implementation over two hundred random instances, agreeing on all of them, together with the boundary cases of a perfect ranking, a worst ranking, a user with no relevant items, and correct exclusion of items already seen in training.

3.7 Hardened verification

The code is clean under a strict warning set including implicit conversion and sign conversion diagnostics. Beyond that, the whole suite runs under a hardened build, available as a build target so that it is repeatable rather than a one-off.

The compiler on this machine ships no sanitizer runtime library, so the usual address and undefined-behaviour sanitizers cannot link. Undefined-behaviour checking is still available in *trap* mode, where a violation compiles to an illegal instruction rather than a call into a diagnostic runtime, and that needs no library. The suite is clean under it.

A clean run under an instrumentation that silently did nothing would look identical to a clean run under one that worked, so both instrumentations were verified against deliberate faults. A program performing an out-of-range shift runs to completion and prints a nonsense value when uninstrumented, and dies on an illegal instruction when instrumented. The bounds check described below behaves the same way.

Checking the standard library covers containers and iterators, but it does nothing for a hand-written type. That mattered here. The local span type is the only unchecked raw-pointer indexing anywhere in this codebase, and it carries the deserialisation path, which parses attacker-controlled bytes. The hardened build was therefore leaving precisely the most exposed path unverified. The span now performs its own bounds check under the same macros, reporting the offending index, the size and the source location before aborting. Under a release build it costs nothing.

The remaining instrumentation gap is a genuine address sanitizer, which would catch heap buffer overflows, use after free and leaks. Its principal failure classes do not arise here, because the code performs no manual heap management at all: there is no occurrence of `new`, `delete`, `malloc` or `free` in the source, and every owning container is a standard vector. That is a weaker statement

than having run the tool, and it is recorded as such rather than presented as equivalent.

References